진행 보고: 문자 단위 CNN 내부 조기 종료가 단독 모델보다 정확도–속도 그래프에서 위에 섰습니다

학부 졸업논문 · 보고일 2026-10-25 · 직전 보고(2026-10-14) 이후분

1. 요약

직전 보고 15절에서 등록한 딥러닝 후보 세 가지 중 후보 B(앞에 더 가벼운 문자 모델)와 후보 C(문자 단위 CNN 내부의 조기 종료)를 측정했습니다. 학습 없이 상한부터 재는 관문(2절)을 두 후보 모두 통과해, 판정 기준을 먼저 저장소에 적어 둔 뒤(3절) 학습과 본 측정을 했습니다(4절).

- **후보 C 는 미리 정한 세 조건을 모두 채웠습니다.** 실제 사용자 요청 기준으로 문자 단위 CNN 단독보다 **3.53배** 빠르고 Macro-F1 은 0.10%p 낮습니다(허용폭 0.3%p 이내). 정해 주신 기준으로 "문자 단위 CNN 단독보다 정확도–속도 그래프에서 위"라고 쓸 수 있는 첫 구성입니다.
- **후보 B 는 위가 아닙니다.** 2.39배 빨랐지만 Macro-F1 이 0.45%p 낮아 허용폭을 넘었습니다.
- **다만 후보 C 의 이득은 "넘김 판단"에서 나오지 않았습니다(5절).** 다섯 fold 중 세 개는 요청을 하나도 넘기지 않았고, 두 fold 에서는 보조 출력 혼자가 문자 단위 CNN 전체보다 정확했습니다. 정직하게 읽으면 "문자 단위 CNN 이 이 데이터에 필요 이상으로 크다"는 결과입니다.
- **같은 날 후속 측정(6절):** 같은 크기 모델을 혼자 학습하면 1.36%p 낮아 미리 정한 조건을 못 채웠고(단, 혼자 학습한 쪽이 불안정했고 차이는 유의하지 않음), 난수 씨앗을 바꾼 확인에서도 후보 C 는 3.62배 ·+0.33%p 로 다시 위였습니다(2/2). 씨앗 43 에서는 실제 사용자 요청을 하나도 넘기지 않아, 후보 C 의 본질은 넘김이 아니라 **함께 학습해 얻은 작은 출구**입니다.

이 보고서의 용어는 직전 보고와 같습니다. "속도 배수"는 문자 단위 CNN 단독의 요청당 지연을 비교 구성의 지연으로 나눈 값으로, 1보다 크면 그 구성이 빠릅니다. 별도 표시가 없으면 공격 비율 3.3% 로 환산했고, 지연은 직전 보고 5절의 공정한 기준(전처리 포함, 실제 길이, CPU 스레드 1개, 요청 하나씩)입니다.

2. 관문: 문자 단위 CNN 의 시간은 어디에 쓰이는가

후보 B 와 C 는 모두 "문자 단위 CNN 의 일부만 돌려서 시간을 아낀다"는 생각입니다. 만약 모델 크기와 무관한 고정 비용(입력 변환, 호출 준비)이 시간의 대부분이라면 모델을 줄여도 빨라지지 않으므로, 학습 전에 문자 단위 CNN 의 순전파를 단계별로 나눠 쟀습니다.

단계	요청당 지연 (ms, 중앙값)
입력 변환(문자열 → 바이트 번호)	0.0093
임베딩	0.0076
합성곱(폭 3)	0.0808
합성곱(폭 5)	0.1075
합성곱(폭 7)	0.1388
분류기	0.0123
문자 단위 CNN 전체(입력 변환 포함)	0.361
가장 싼 신경망(임베딩 → 평균 → 선형, 정확도 없는 하한)	0.0214

검증 데이터 2,000건, 30회 반복, CPU 스레드 1개. 단계별 합과 전체 순전파의 차이는 0.005ms 로, 나눠 재는 것이 측정을 왜곡하지 않았습니다.

고정 비용은 전체의 8.5% 뿐이고 **91% 가 세 합성곱**이었습니다. 그래서 모델을 줄이면 실제로 빨라집니다. 넘김 비율을 5.3% 로 가정한 상한은 후보 B 가 7.24배, 후보 C(폭 3 합성곱까지만 돌리고 보조 출력으로 판정)가 3.03배로, 미리 정한 관문 1.5배를 둘 다 넘었습니다. 다만 상한은 상한일 뿐이고, 실제 배수는 작은 모델이나 보조 출력이 **몇 %를 넘기면서 단독 모델의 정확도를 지키는가**로 정해지므로 학습이 필요했습니다.

3. 본 측정의 판정 기준 (학습 전에 고정)

학습을 시작하기 전에 구조, 학습 설정, 판정 기준을 저장소에 커밋했습니다. 결과를 보고 바꾸지 않았습니다.

- **후보 B 의 1차:** 문자 단위 CNN 과 같은 꼴에서 임베딩 32차원, 폭 3 합성곱 하나, 필터 64개. 2차는 기존 문자 단위 CNN 입니다.
- **후보 C:** 문자 단위 CNN 과 같은 구조에 폭 3 합성곱의 출력 위로 보조 출력(선형 층 하나)을 답니다. 학습은 "본 출력 손실 + 0.3 × 보조 출력 손실"을 함께 줄이는 방식으로, 이전 단계에서 이미지 CNN 의 조기 종료에 쓴 것과 같습니다. 보조 출력이 확신하면 거기서 끝내고, 아니면 나머지 두 합성곱과 분류기를 돌립니다. 앞에서 한 계산을 버리지 않습니다.
- **학습과 평가:** 시험 데이터를 뺀 226,048건의 5겹 교차검증, 기존 문자 단위 CNN 과 같은 설정과 같은 fold. 넘김 임계값은 fold 마다 **검증 데이터에서** 고릅니다(검증 Macro-F1 이 단독 모델보다 0.5%p 이상 낮지 않은 점 중 넘김이 가장 적은 점).
- **판정(후보마다, 실제 사용자 요청 기준):** ① fold 평가 속도 배수 평균 1.2배 이상(지연 측정의 날 간 흔들림 8% 를 넘는 크기) ② 다섯 fold 모두 1배 초과 ③ Macro-F1 평균이 단독 모델보다 0.3%p 넘게 낮지 않음. 셋을 다 채우면 "위", 하나라도 못 채우면 "위가 아니다"로 적습니다.

4. 본 측정 결과

구성	정상 요청 기준	속도 배수 평균 (fold 범위)	Macro-F1 (단독 대비)	넘김: 정상 · 공격	판정
후보 B: 작은 문자 모델 → 문자 단위 CNN	실제 사용자	2.39 (1.52~4.10)	0.9839 (−0.45%p)	30.2% · 4.0%	**위가 아니다** (③ 실패)
	검증 데이터 전체	4.41 (3.16~5.19)	0.9839 (−0.45%p)	6.0% · 4.0%	(병기) ③ 실패
후보 C: 문자 단위 CNN 내부 조기 종료	실제 사용자	**3.53** (3.24~3.62)	0.9874 (−0.10%p)	1.1% · 0.4%	**위** (세 조건 충족)
	검증 데이터 전체	3.60 (3.54~3.62)	0.9874 (−0.10%p)	0.2% · 0.4%	(병기) 세 조건 충족

비교 대상 문자 단위 CNN 단독의 Macro-F1 은 0.9884, 속도 배수는 1. 지연(같은 실행, 중앙값): 문자 단위 CNN 전체 0.345ms, 작은 문자 모델 0.061ms, 후보 C 의 앞부분(폭 3 합성곱 + 보조 출력) 0.095ms, 나머지 0.255ms.

- **후보 B** 의 작은 모델 혼자의 Macro-F1 은 0.9601 로 낮았습니다. 단독 모델의 정확도를 지키려면 정상 요청의 30% 를 넘겨야 했고, 그렇게 넘겨도 0.45%p 가 모자랐습니다. 검증 데이터에서 허용한 0.5%p 를 거의 다 쓰고 평가 데이터에서 그만큼 잃은 모양입니다.
- **후보 C** 의 보조 출력 혼자의 Macro-F1 은 0.9849, 본 출력 혼자는 0.9898 이었습니다. 함께 학습한 것이 본 출력의 정확도를 깎지 않았습니다(단독 모델보다 0.14%p 높지만 씨앗 잡음 범위입니다).
- 클래스별 임계값을 같은 규칙으로 병기하면 후보 B 가 5.17배, Macro-F1 −0.28%p 로 숫자상 세 조건을 채웁니다. 그러나 미리 판정 대상으로 정한 것이 아니어서 이것으로 "위"라고 쓰지는 않겠습니다.

5. 해석: 후보 C 의 이득은 어디서 왔는가

후보 C 의 결과를 fold 별로 다시 열어 보았습니다.

fold	단독 모델 Macro-F1	보조 출력 혼자	후보 C	차이	실제 사용자 넘김
1	0.9846	0.9920	0.9920	+0.74%p	0%
2	0.9929	0.9891	0.9894	−0.36%p	0.9%
3	0.9887	0.9682	0.9801	−0.86%p	4.5%
4	0.9848	0.9895	0.9896	+0.48%p	0%
5	0.9908	0.9857	0.9858	−0.50%p	0%

다섯 fold 중 세 개에서 검증 데이터가 고른 운영점은 **넘김 0%**, 즉 보조 출력 혼자 판정하는 것이었습니다. fold 1 과 4 에서는 폭 3 합성곱 하나 위의 보조 출력이 문자 단위 CNN 전체보다 정확했습니다. fold 3 은 학습이 2 에폭에서 멈춰 보조 출력이 덜 학습됐고(0.9682), 넘김으로도 손실을 다 메우지 못했습니다.

따라서 이 결과가 지지하는 문장은 "넘김 판단을 고쳐 속도를 얻었다"가 아니라 **"문자 단위 CNN 은 이 데이터에 필요 이상으로 크고, 함께 학습한 폭 3 합성곱 하나로 거의 같은 정확도를 약 3.6배 빠르게 낸다"**입니다. 넘김은 덜 학습된 fold 를 일부 메우는 보조 역할만 했습니다.

이렇게 읽으면 바로 다음 질문이 나옵니다. **폭 3 합성곱 하나짜리 문자 모델을 처음부터 혼자 학습해도 같은 결과가 나오는가.** 같다면 후보 C 는 "작은 모델 하나"로 환원되고, 조기 종료 구조의 몫은 없습니다. 후보 B 의 작은 모델은 임베딩과 필터가 절반이라 이 질문의 답이 되지 못합니다.

6. 후속 측정 결과 (같은 날 진행)

5절에서 나온 질문과 씨앗 확인, 비교 그래프를 판정 기준을 먼저 저장소에 적은 뒤(7a1f725) 진행했습니다. 학습 설정과 지연 값은 4절과 같습니다.

6.1 폭 3 합성곱 하나짜리 문자 모델을 혼자 학습하면

후보 C 의 보조 출력과 같은 크기(임베딩 64차원, 폭 3 합성곱, 필터 128개)의 모델을 처음부터 혼자 학습했습니다. 계산 경로가 후보 C 의 앞부분과 같아 속도는 3.62배로 같습니다. 이 모델에 4절과 같은 세 조건을 걸었습니다.

구성 (씨앗 42)	속도 배수	Macro-F1 (단독 대비)	판정
문자 단위 CNN 단독	1	0.9884	비교 대상
후보 C (함께 학습한 보조 출력 + 넘김)	3.53	0.9874 (−0.10%p)	위
폭 3 모델 혼자 학습	3.62	0.9748 (−1.36%p)	**위가 아니다** (정확도 조건 실패)
폭 3 모델 → 문자 단위 CNN (넘김, 병기)	2.75	0.9830 (−0.53%p)	위가 아니다

같은 크기를 혼자 학습하면 1.36%p 낮아, 미리 적어 둔 규칙대로 **"후보 C 의 결과는 작은 모델 하나로 환원되지 않는다. 함께 학습한 것(또는 넘김)의 몫이 있다"**로 기록했습니다. 다만 두 가지를 함께 밝혀 둡니다.

- 후보 C 의 보조 출력과 이 모델의 fold 별 차이는 평균 +1.01%p 지만 fold 마다 −1.36 ~ +2.04%p 로 흔들려, 보정 t-검정으로 유의하지 않습니다(p 0.34).
- 혼자 학습한 모델은 **학습이 불안정했습니다.** 다섯 fold 중 넷의 최적 에폭이 2~9 에 머물렀고(후보 C 는 씨앗 43 에서 12~19), 검증 Macro-F1 이 에폭 사이에 최대 5%p 오르내렸습니다. 따라서 "함께 학습한

몫"이 표현력이 아니라 학습 안정성일 수 있고, 학습률이나 조기 종료 기준을 바꾸면 차이가 줄 수도 있습니다. 이번 설계로는 가를 수 없습니다.

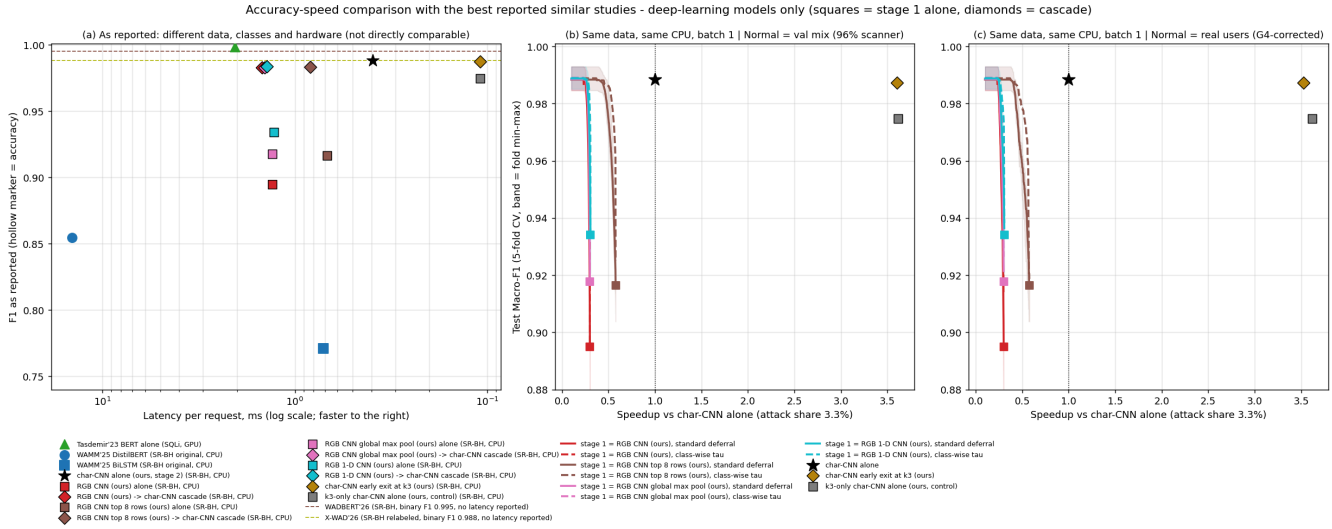
6.2 난수 씨앗을 바꾼 확인

후보 C 와 비교 대상(문자 단위 CNN 단독)을 씨앗 43 으로 다시 학습해 같은 규칙을 적용했습니다.

씨앗	단독 모델 Macro-F1	후보 C Macro-F1 (차이)	속도 배수 (fold 범위)	실제 사용자 넘김	판정
42	0.9884	0.9874 (−0.10%p)	3.53 (3.24~3.62)	1.1%	위
43	0.9862	0.9895 (+0.33%p)	3.62 (모든 fold 3.62)	0.0%	위

두 씨앗 모두 세 조건을 채웠습니다(2/2). 씨앗 두 개라 "견고하다"고까지는 쓰지 않겠습니다. 눈여겨볼 점은 씨앗 43 에서는 **다섯 fold 모두 넘김이 0%**였다는 것입니다. 실제로 배포되는 것은 "함께 학습한 폭 3 합성곱과 보조 출력"뿐이고, 나머지 두 합성곱은 한 번도 돌지 않습니다.

6.3 유사 연구 비교 그래프



딥러닝 모델만 남긴 정확도–속도 비교(직전 보고 15절의 그림에 두 점을 더함). 황토색 마름모가 후보 C, 회색 네모가 폭 3 모델 혼자 학습, 검은 별이 문자 단위 CNN 단독입니다. (a)는 각 논문에 적힌 값 그대로라 데이터·클래스·기기가 달라 직접 비교가 되지 않고, (b)(c)는 같은 데이터·같은 CPU 에서 잰 값입니다.

(a)에서 후보 C 는 같은 SR-BH 데이터를 쓴 유사 연구의 딥러닝 보고값(BiLSTM 0.713ms·0.77, DistilBERT 14.4ms·0.85)보다 빠르고 높은 자리에 찍힙니다. 그러나 그 연구와 데이터 정제·클래스 구성·기기가 달라 이것으로 "유사 연구보다 개선"이라고 쓰지는 않겠습니다. 미리 적어 둔 대로 이 그림은 위치를 보여 주는 용도입니다.

6.4 정리

세 측정을 합치면 후보 C 의 결과는 "넘김 판단을 고쳐 속도를 얻었다"가 아니라 **"문자 단위 CNN 을 폭 3 보조 출력과 함께 학습하면, 그 보조 출력 하나로 단독 모델과 같은 정확도를 약 3.5배 빠르게 낸다"**입니다. 즉 2차 모델을 압축하는 방법이고, 논문의 두 축 중 넘김 판단 개선이 아니라 속도–정확도 교환 쪽에 속합니다. 캐스케이드 구성들을 비교할 때 "캐스케이드 없이도 얻을 수 있는 기준선"으로 쓰는 것이 정직하다고 생각합니다. 논문 본문에 어떻게 넣을지는 말씀을 듣고 정하겠습니다.

7. 다음에 할 것

1. **혼자 학습한 폭 3 모델의 학습 안정화 대조.** 학습률을 낮추거나 조기 종료 기준을 늘려, 6.1 의 차이가 표현력 때문인지 학습 안정성 때문인지 가릅니다. 판정 기준을 먼저 적은 뒤 진행합니다.
2. **후보 A(문자 단위 CNN → 트랜스포머)**는 아직 관문(트랜스포머 단독이 문자 단위 CNN 단독보다 정확한가)을 재지 않았습니다.
3. 후보 C 를 논문에서 어느 축에 둘지(6.4)에 대한 의견을 여쭙고 싶습니다.

참고: 재현 정보

측정 기기: AMD Ryzen 7 9800X3D(8코어 16스레드), RAM 32GB, NVIDIA RTX 4080 SUPER, Windows 11, PyTorch 2.6.0(CUDA 12.4). 측정일 2026-10-25. 지연은 검증 데이터에서 뽑은 2,000건을 30회 반복한 중앙값이고, 학습이 끝난 뒤 다른 작업이 돌지 않는 상태에서 쟀습니다. 같은 실행 안의 값끼리만 배수를 계산했습니다(문자 단위 CNN 전체는 관문 측정에서 0.361ms, 본 측정에서 0.345ms 로 측정일에 따라 4% 가량 다릅니다). 판정 기준은 학습 전에(4절 f5a002a · 6절 7a1f725), 판정 코드는 결과가 나오기 전에(4c89b3e · 8882b5c) 저장소에 커밋했습니다. 6절의 지연은 4절 측정값을 그대로 썼습니다.